Searching & Sorting

a. Linear Search: Algo: traverse through the array one by one if found return index otherwise -1.

```
function linearSearch(arr, target)
{
    for(let i=0 ; i< arr.length; i++)
    {
        if (arr[i] === target){
            return i;
        }
    }
    return -1;
}
```

[4, 9, 1, 0, 2]

i	target	arr[i]
0	0	4 X
1		9 X
2		1 X
3		0 ✓

o/p: 3

b. Binary Search: Algo:

1. Keep three pointers left, right and middle
   middle = (left + right)/2
2. Check if middle element is target if it is a target return middle.
3. else check if target > middle if it is Change left to middle + 1
4. else change right to middle - 1
5. Do 2, 3, 4 until right >= left

Note: Array should be Sorted already.

Time complexity : O(log n)
Space complexity : O(1)

$n \to n/2 \to n/4 \cdots = 1$

$n/2^x = 1 \to \boxed{x = \log_2 n}$

Code:

```
function binarySearch(arr, target)
{
    let left = 0
    let right = arr.length - 1;
    while (right >= left)
    {
        let middle = Math.floor( (left+right)/2);
        if ( arr[middle] === target )
            return middle;
        else if ( target > arr[middle])
            left = middle + 1;
        else
            right = middle - 1;
    }
    return -1;
}
```

Dry run:

arr = [0, 1, 2, 5, 7, 8, 9]
(0 1 2 3 4 5 6)

left = 0, right = 6, target = 11

left	right	middle	target	if	else if	else
0	6	3	11	X	✓	not executed
4	6	5	11	X	✓	not executed
6	6	6	11	X	✓	not executed
7	6	6	11			

right >= left (X fails)

returns -1.

Bubble Sort

arr = [9, 8, 12, 11, 7, 2, 3, 1] = 8 $\quad$ n = 8

Algo: 1) compare in pairs make the largest element go to the last place.

2) then second largest, third largest .... until sorted.

1st iteration: (largest element at end)

[(9, 8) 12, 11, 7, 2, 3, 1]

[8, (9, 12) 11, 7, 2, 3, 1]

[8, 9, (12, 11) 7, 2, 3, 1]

[8, 9, 11, (12, 7) 2, 3, 1]

[8, 9, 11, 7, (12, 2) 3, 1]

[8, 9, 11, 7, 2, (12, 3) 1]

[8, 9, 11, 7, 2, 3, (12, 1)]

[8, 9, 11, 7, 2, 3, 1 12]

is swapped = true

2nd iteration : (second largest)

[(8, 9) 11, 7, 2, 3, 1, 12]

[8, (9, 11) 7, 2, 3, 1, 12]

[8, 9, (11, 7) 2, 3, 1, 12]

[8, 9, 7, (11, 2) 3, 1, 12]

[8, 9, 7, 2, (11, 3) 1, 12]

[8, 9, 7, 2, 3, 11, 1, 12]

[8, 9, 7, 2, 3, 1, 11, 12]

is swapped = true

3rd iteration: [8, 9, 7, 2, 3, 1, 11, 12]

[8, 9, 7, 2, 3, 1, 11, 12]

[8, 7, 9, 2, 3, 1, 11, 12]

[8, 7, 2, 9, 3, 1, 11, 12]

[8, 7, 2, 3, 9, 1, 11, 12]

[8, 7, 2, 3, 1, 9, 11, 12]

is Swapped = true

4th Iteration: [8, 7, 2, 3, 1, 9, 11, 12]

[7, 8, 2, 3, 1, 9, 11, 12]

[7, 2, 8, 3, 1, 9, 11, 12]

[7, 2, 3, 8, 1, 9, 11, 12]

[7, 2, 3, 1, 8, 9, 11, 12]

is Swapped = true

5th Iteration : [7, 2, 3, 1, 8, 9, 11, 12]

[2, 7, 3, 1, 8, 9, 11, 12]

[2, 3, 7, 1, 8, 9, 11, 12]

6th Iteration:

[2, 3, 1, 7, 8, 9, 11, 12]

is swapped = false

[2, 3, 1, 7, 8, 9, 11, 12]

[2, 1, 3, 7, 8, 9, 11, 12] is swapped = true

[2, 3, 1, 7, 8, 9, 11, 12]

7th Iteration: [2, 1, 3, 7, 8, 9, 11, 12]

[1, 2, 3, 7, 8, 9, 11, 12]

is swapped

Conclusion: n=8

1. need (n-1) iterations → 7
2. for each iteration we swapped/compared for (n-i-1)
   (7, 6, 5, 4, 3, 2, 1) → happened like this
3. tracking swapped or not helpful to break the loop if the swap did not happen in any iterations

O(n²) – time
O(1) – space

Code:

```
function bubblesort (arr)
{
   let n = arr.length;
   let isSwapped = false;
   for(let i=0 ; i<(n-1) ; i++)
   {
     for(let j=0; j<(n-i-1) ; j++)
     {
          if ( arr[j] > arr[j+1])
          {
               let temp = arr[j] ;
               arr[j] = arr[j+1];
               arr[j+1] = temp;
               isSwapped = true;
          }
     }
     if (!isSwapped) break;
   }
   return arr;
}
```

Selection Sort

⇒ selecting the minimum element and putting that in a Correct place.

Ex: [9, 2, 3, 1, 5] n=5

index i, min

→ lets assume minimum index is 0

→ from index+1 we will check minimum if exists we update.

→ then finally swap to front.

(i) 1st Iteration:

[1, 2, 3, 9, 5]

(i) 2nd Iteration: min and i are same no swap

(i) 3rd Iteration: [1, 2, 3, 9, 5] again min and i are same no swap

(i) 4th Iteration : [1, 2, 3, 9, 5] not same and smaller than i so swap min and i

min

[1, 2, 3, 5, 9]

Conclusion:

(1) for n elements we did n-1 iterations
(2) In each iteration we started from i+1 till n
(3) swapped minimum with i

$O(n^2)$ - time
$O(1)$ - space

Code:

```
function selectionSort(arr)
{
    let n = arr.length;
    for(i=0; i<n-1 ; i++)
    {
        let min = i;
        for(j= i+1; j<n ; j++)
        {
            if (arr[j] < arr[min]){
                min = j;
            }
        }
        if ( min !== i)
        {
            let temp = arr[min]
            arr[min] = arr[i]
            arr[i] = temp
        }
    }
    return arr;
}
```

Insertion Sort

arr = [4, 5, 1, 3, 9]
prev curr

[4, 5, 1, 3, 9]
prev curr

[1 4 5 3 9]
prev curr

[1 3 4 5 9]
prev curr

[1 3 4 5 9]

Algo: (1) Consider first element is sorted after that element all are unsorted
(2) we need to traverse from 1 to n
(3) track current and previous values

current is i
previous is i-1

(4) if we find current value is smaller than previous value!
⇒ move previous value to next
⇒ update previous value to one less than the current previous value.
(5) repeat 4 until we find the previous value

greater than curr and make sure previous is not out of bound

⑥ Then shift the current value to prev+1

Code:

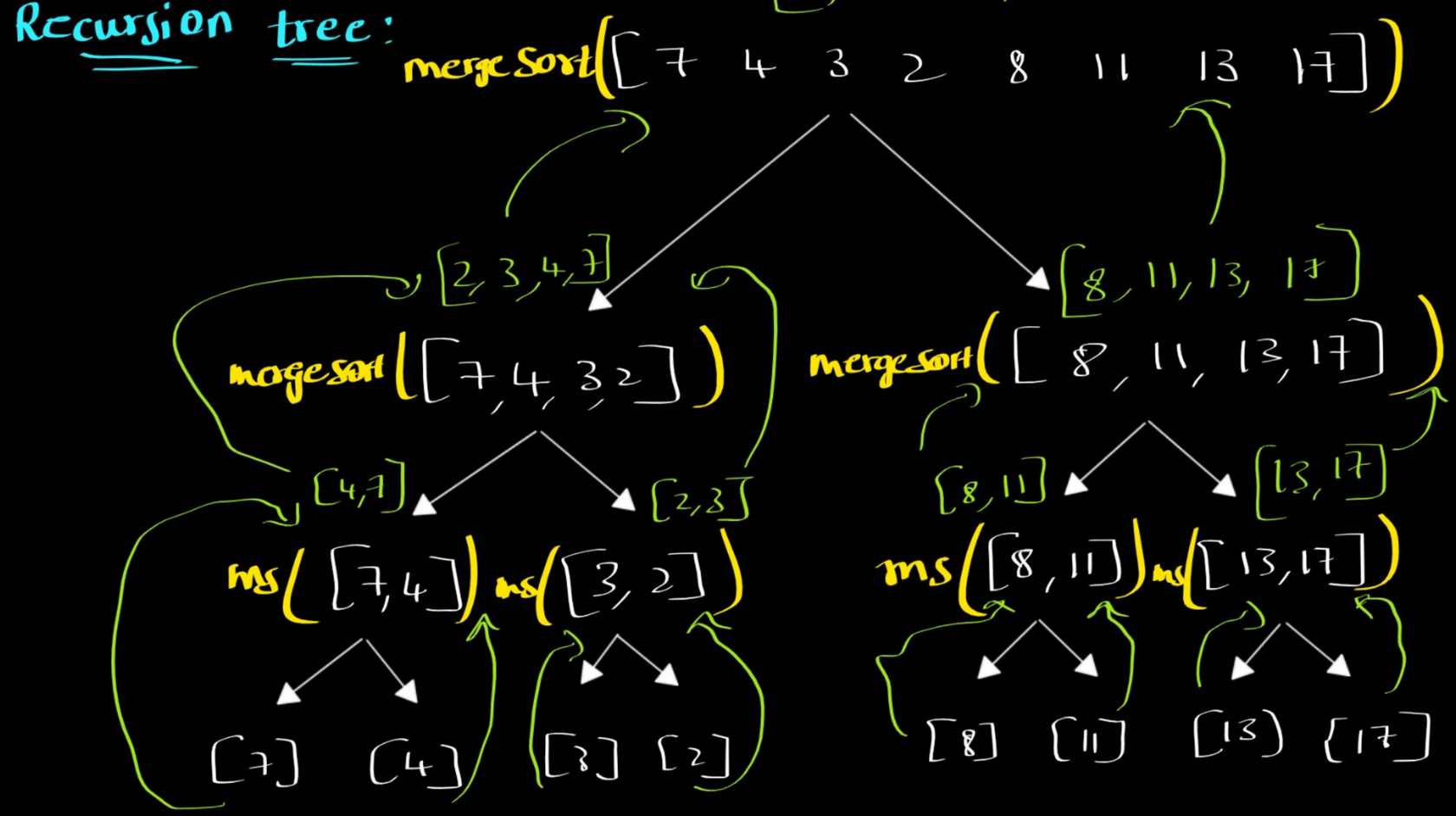
```
function insertionSort(arr)
{
    let n = arr.length;
    for(let i = 1; i < n; i++)
    {
        let curr = arr[i];
        let prev = i-1;
        while( arr[prev] > arr[curr] && prev >= 0)
        {
            arr[prev+1] = arr[prev];
            prev--;
        }
        arr[prev+1] = curr;
        return arr;
}
```

Merge Sort

arr = [7 4 3 2 8 11 13 17]

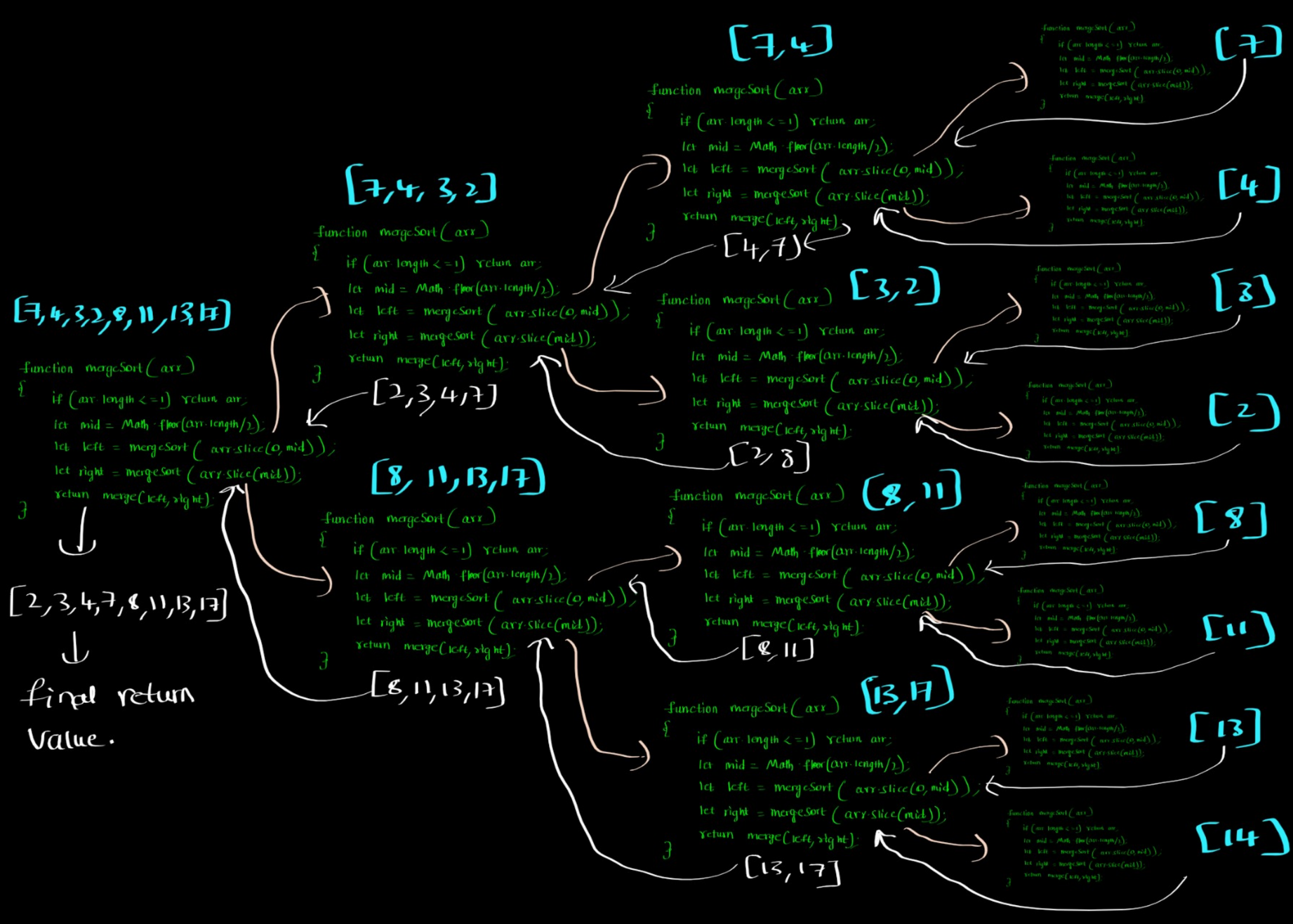
[2, 3, 4, 7, 8, 11, 13, 17] → final answer

Recursion tree:



Code:

```
function mergeSort(arr)
{
    if(arr.length <= 1) return arr;
    let mid = Math.floor(arr.length/2);
    let left = mergeSort(arr.slice(0, mid));
    let right = mergeSort(arr.slice(mid));
    return merge(left, right);
}
```



Code:

```
function mergeSort(arr)
{
    if (arr.length <= 1) return arr;
    let mid = Math.floor(arr.length/2);
    let left = mergeSort(arr.slice(0, mid));
    let right = mergeSort(arr.slice(mid));
    return merge(left, right);
}
```

$n \to n/2 \to n/4 \cdots 1$

$n/2^x = 1$

$\log(n) = \log 2^x$

$x = \log(n)$

$O(\log(n))$

```
function merge(left, right)
{
    let result = [ ];   ⟶ As we are creating new array
    let ptr1 = 0;
    let ptr2 = 0;
    while(ptr1 <= left.length-1 && ptr2 <= right.length-1)
    {
        if(left[ptr1] < right[ptr2])
        {
            result.push[left[ptr1]]
            ptr1++;
        }
        else {
            result.push[right[ptr2]]
        }
    }
    return [...res, ...left.slice(ptr1), ...right.slice(ptr2)];
```

Space complexity → O(n)

O(n)

Time complexity

$O(n \log(n))$

$O(\log(n)) \times O(n)$

Algo:
1. Keep on dividing the array into two parts until length of array becomes 1
2. As it is single element it is already sorted so return it.
3. we will use helper function merge to sort the left and right part and return it.
4. we will keep on merging the sorted array and return it
5. finally we get the completely sorted array.

Note: It is a divide and conquer algorithm.

> divide → dividing the array into left and right parts
>
> Conquer → merging the sorted left and right parts and returning it back.